



String Permutations by changing case (medium)

We'll cover the following

- Problem Statement
- Try it yourself
- Solution
- Code
 - Time complexity
 - Space complexity

Problem Statement

Given a string, find all of its permutations preserving the character sequence but changing case.

Example 1:

Input: "ad52"

Output: "ad52", "Ad52", "aD52", "AD52"

Example 2:

Input: "ab7c"

Output: "ab7c", "Ab7c", "aB7c", "AB7c", "ab7C", "Ab7C", "aB7C", "AB7C"

Try it yourself

Try solving this question here:

Java

Python3

JS

C++

```
1 import java.util.*;
2
3 class LetterCaseStringPermutation {
4
5     public static List<String> findLetterCaseStringPermutations(String str) {
6         List<String> permutations = new ArrayList<>();
7         // TODO: Write your code here
8         return permutations;
9     }
10
11     public static void main(String[] args) {
12         List<String> result = LetterCaseStringPermutation.findLetterCaseStringPermutations("ad52");
13         System.out.println(" String permutations are: " + result);
14
15         result = LetterCaseStringPermutation.findLetterCaseStringPermutations("ab7c");
16         System.out.println(" String permutations are: " + result);
17     }
18 }
```



```

16 }
17 }
18 }
19 }

```

Run

Save

Reset



Solution

This problem follows the [Subsets](#) pattern and can be mapped to [Permutations](#).

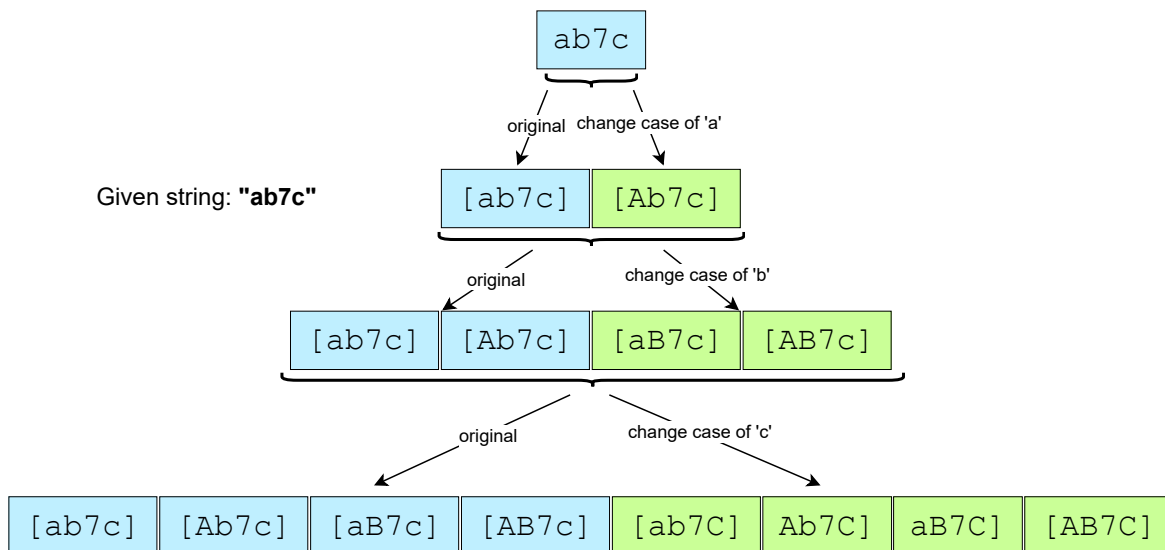
Let's take Example-2 mentioned above to generate all the permutations. Following a **BFS** approach, we will consider one character at a time. Since we need to preserve the character sequence, we can start with the actual string and process each character (i.e., make it upper-case or lower-case) one by one:

1. Starting with the actual string: "ab7c"
2. Processing the first character ('a'), we will get two permutations: "ab7c", "Ab7c"
3. Processing the second character ('b'), we will get four permutations: "ab7c", "Ab7c", "aB7c", "AB7c"
4. Since the third character is a digit, we can skip it.
5. Processing the fourth character ('c'), we will get a total of eight permutations: "ab7c", "Ab7c", "aB7c", "AB7c", "ab7C", "Ab7C", "aB7C", "AB7C"

Let's analyze the permutations in the 3rd and the 5th step. How can we generate the permutations in the 5th step from the permutations in the 3rd step?

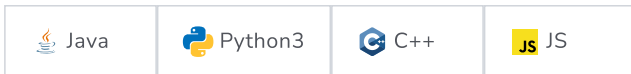
If we look closely, we will realize that in the 5th step, when we processed the new character ('c'), we took all the permutations of the previous step (3rd) and changed the case of the letter ('c') in them to create four new permutations.

Here is the visual representation of this algorithm:



Code

Here is what our algorithm will look like:



```

1  import java.util.*;
2
3  class LetterCaseStringPermutation {
4
5      public static List<String> findLetterCaseStringPermutations(String str) {
6          List<String> permutations = new ArrayList<>();
7          if (str == null)
8              return permutations;
9
10         permutations.add(str);
11         // process every character of the string one by one
12         for (int i = 0; i < str.length(); i++) {
13             if (Character.isLetter(str.charAt(i))) { // only process characters, skip digits
14                 // we will take all existing permutations and change the letter case appropriately
15                 int n = permutations.size();
16                 for (int j = 0; j < n; j++) {
17                     char[] chs = permutations.get(j).toCharArray();
18                     // if the current character is in upper case change it to lower case or vice versa
19                     if (Character.isUpperCase(chs[i]))
20                         chs[i] = Character.toLowerCase(chs[i]);
21                     else
22                         chs[i] = Character.toUpperCase(chs[i]);
23                     permutations.add(String.valueOf(chs));
24                 }
25             }
26         }
27         return permutations;
28     }
29
30     public static void main(String[] args) {
31         List<String> result = LetterCaseStringPermutation.findLetterCaseStringPermutations("ad52");
32         System.out.println(" String permutations are: " + result);
33
34         result = LetterCaseStringPermutation.findLetterCaseStringPermutations("ab7c");
35         System.out.println(" String permutations are: " + result);
36     }
37 }
38

```

Run

Save

Reset



Time complexity

Since we can have 2^N permutations at the most and while processing each permutation we convert it into a character array, the overall time complexity of the algorithm will be $O(N * 2^N)$.

Space complexity

All the additional space used by our algorithm is for the output list. Since we can have a total of $O(2^N)$ permutations, the space complexity of our algorithm is $O(N * 2^N)$.

[< Back](#)
☒ Mark As Completed

[Next >](#)

Permutations (medium)

Balanced Parentheses (hard)



